

ITI123 Project Milestone Report

Badminton Stroke Classification using Pose Estimation: Clear vs Smash Stroke Analysis

Student Name
Student ID
ITI123 - Artificial Intelligence

January 15, 2026

Abstract

This milestone report presents a comprehensive investigation into classifying badminton Clear and Smash strokes using pose estimation techniques. We implemented MediaPipe-based pose extraction on the ShuttleSet dataset (4,983 clips, 40 matches) and developed a machine learning pipeline with multiple model architectures. Despite implementing state-of-the-art feature engineering including novel wrist orientation features, comprehensive data preprocessing with group-stratified splitting, and proper padding masking for sequence models, all classifiers achieved approximately 50% accuracy (random guessing level). Through rigorous Cohen's d effect size analysis, we identified the fundamental issue: Clear and Smash strokes are biomechanically indistinguishable at shuttlecock contact using pose-only features (Cohen's d ≤ 0.2 across all 60 features). This report documents our methodology, extensive analysis, improvement iterations, and provides evidence-based recommendations for future work requiring multi-modal sensing or alternative stroke pair selection.

Contents

1	Introduction	3
1.1	Problem Statement	3
1.2	Dataset	3
1.3	Methodology Overview	3
2	Technical Implementation	4
2.1	Pose Extraction	4
2.2	Feature Engineering	4
2.2.1	Version 1 (Baseline Features)	4
2.2.2	Version 2 (Enhanced with Wrist Orientation)	4
2.3	Data Splitting	5
2.3.1	Group-Stratified Split	5
2.3.2	Padding Masking	6
2.4	Model Architectures	7

2.4.1	Baseline Models	7
2.4.2	Deep Learning Models	7
3	Results and Analysis	8
3.1	Model Performance	8
3.2	Cohen’s d Effect Size Analysis	9
3.2.1	Overall Feature Analysis	9
3.2.2	Top 10 Features by Effect Size	9
3.2.3	Wrist Orientation Features Analysis	10
3.3	Visualization of Class Overlap	10
4	Discussion	10
4.1	Why Classification Failed	10
4.1.1	Biomechanical Analysis	10
4.1.2	What Actually Differs (Not Captured)	11
4.2	Methodological Contributions	11
4.3	Limitations	12
4.3.1	Dataset Limitations	12
4.3.2	MediaPipe Limitations	12
5	Recommendations	12
5.1	For Clear vs Smash Classification	12
5.1.1	Option 1: Add Racket Tracking	12
5.1.2	Option 2: Add Shuttlecock Tracking	13
5.1.3	Option 3: Extend Temporal Window	13
5.1.4	Option 4: Multi-Modal Fusion	13
5.2	Alternative Stroke Pairs (Pose-Only)	13
6	Conclusion	14
A	Code Repository Structure	15
B	Feature List	15
B.1	All 60 Sequence Features	15

1 Introduction

1.1 Problem Statement

Badminton stroke classification is crucial for automated coaching systems, performance analysis, and player training. This project focuses on distinguishing between two fundamental overhead strokes:

- **Clear:** Defensive stroke sending the shuttlecock high to the opponent’s backcourt
- **Smash:** Aggressive attacking stroke with steep downward trajectory

The primary objective is to investigate whether **pose estimation alone** can reliably classify these strokes.

1.2 Dataset

Source: ShuttleSet - A Human Action Recognition Dataset for Badminton Singles
Statistics:

- Total clips: 4,983 (extracted at shuttlecock contact)
- Clear strokes: 2,660 (53.4%)
- Smash strokes: 2,321 (46.6%)
- Unique matches: 40
- Pose extraction success rate: 99.4% (4,954/4,983)

1.3 Methodology Overview

Our approach follows a systematic pipeline:

1. Pose extraction using MediaPipe
2. Feature engineering (spatial, temporal, and wrist orientation features)
3. Data splitting with match-level stratification
4. Model training (traditional ML and deep learning)
5. Statistical validation using Cohen’s d effect size
6. Iterative improvements based on diagnostic analysis

2 Technical Implementation

2.1 Pose Extraction

Framework: Google MediaPipe Pose

Output: 33 body landmarks per frame (x, y, z, visibility)

Processing:

Listing 1: MediaPipe Pose Extraction

```
import mediapipe as mp

mp_pose = mp.solutions.pose
pose = mp_pose.Pose(
    static_image_mode=False,
    model_complexity=2, # Highest accuracy
    smooth_landmarks=True,
    min_detection_confidence=0.5
)

# Process video frame-by-frame
for frame in video:
    results = pose.process(frame)
    landmarks = results.pose_landmarks # 33 keypoints
```

Key Landmarks: Shoulders, elbows, wrists, hips, knees, ankles, nose

2.2 Feature Engineering

We implemented two iterations of feature engineering:

2.2.1 Version 1 (Baseline Features)

Features per frame: 42

Table 1: Baseline Feature Categories

Category	Features
Arm Extension	Distance from shoulder to wrist (L/R)
Depth (Z-coordinate)	Wrist/elbow depth relative to shoulder/hip
Height (Y-coordinate)	Wrist/elbow height relative to shoulder/head
Joint Angles	Elbow angle, shoulder angle
Body Posture	Torso lean, shoulder rotation
Velocities	First derivative of all spatial features

2.2.2 Version 2 (Enhanced with Wrist Orientation)

Features per frame: 60

New additions: 9 wrist/forearm orientation features

Table 2: Wrist Orientation Features (New)

Feature	Description
r_forearm_vertical_angle	Angle between forearm and vertical axis (key discriminator hypothesis)
r_forearm_horizontal_angle	Lateral swing direction in X-Z plane
r_wrist_elbow_vertical	Wrist height relative to elbow (flexion indicator)
r_wrist_horizontal_reach	Forward extension distance
r_arm_plane_pronation	Pronation/supination from plane normal
r_arm_plane_vertical	Vertical component of arm plane
r_forearm_pitch	Elevation angle in sagittal plane
l_forearm_vertical_angle	Left arm vertical angle
l_wrist_elbow_vertical	Left wrist-elbow separation

Rationale: Clear and Smash should differ in wrist angle at contact:

- **Clear:** Wrist extended, racket angled upward (expected $\sim 120\text{-}150^\circ$)
- **Smash:** Wrist flexed, racket angled downward (expected $\sim 30\text{-}60^\circ$)

Implementation:

Listing 2: Forearm Vertical Angle Calculation

```
# Forearm vector (elbow to wrist)
forearm_vector = wrist - elbow
forearm_unit = forearm_vector / np.linalg.norm(forearm_vector)

# Vertical axis (downward in image coordinates)
vertical = np.array([0, 1, 0])

# Compute angle
cos_angle = np.dot(forearm_unit, vertical)
vertical_angle = np.arccos(np.clip(cos_angle, -1, 1))
vertical_angle_degrees = np.degrees(vertical_angle)
```

2.3 Data Splitting

2.3.1 Group-Stratified Split

To prevent data leakage and maintain class balance:

Listing 3: Group-Stratified Splitting Strategy

```
# Step 1: Calculate dominant class for each match
for match_id in unique_matches:
    clear_count = count_strokes(match_id, 'Clear')
    smash_count = count_strokes(match_id, 'Smash')
    dominant_class = 'Clear' if clear_count > smash_count else 'Smash'
```

```
# Step 2: Stratify matches by dominant class
clear_matches = matches_with_dominant('Clear')
smash_matches = matches_with_dominant('Smash')

# Step 3: Split each stratum
train_matches = 70% of (clear_matches + smash_matches)
val_matches = 15% of (clear_matches + smash_matches)
test_matches = 15% of (clear_matches + smash_matches)
```

Benefits:

- Prevents data leakage (clips from same match stay together)
- Maintains class balance across splits
- Realistic evaluation (generalizes to new matches)

Results:

Table 3: Data Split Statistics

Split	Clips	Matches	% Clear	% Smash
Train	3,347	27	53.22%	46.78%
Validation	687	5	52.03%	47.97%
Test	920	8	55.10%	44.90%
Overall	4,954	40	53.40%	46.60%

All splits within $\pm 2\%$ of overall distribution.

2.3.2 Padding Masking

Problem: Padded frames pollute normalization statistics

Solution: Normalize only real frames, re-pad with zeros after normalization

Listing 4: Proper Padding Masking

```
# Collect only real frames for normalization
real_frames = []
for seq, length in zip(sequences, seq_lengths):
    real_frames.append(seq[:length]) # Exclude padding

# Compute normalization from real frames only
all_real = np.vstack(real_frames)
mean = np.mean(all_real, axis=0)
std = np.std(all_real, axis=0) + 1e-8

# Normalize and re-pad
for seq, length in zip(sequences, seq_lengths):
    normalized = (seq[:length] - mean) / std
    if length < 50:
        padding = np.zeros((50 - length, n_features))
        normalized = np.vstack([normalized, padding])
```

```
# Save sequence lengths for Masking layer
data = {'X': normalized, 'seq_lengths': seq_lengths}
```

Model Integration:

Listing 5: LSTM with Masking Layer

```
from tensorflow.keras.layers import Masking, LSTM

model = Sequential([
    Masking(mask_value=0.0, input_shape=(50, 60)), # Ignore
        padding
    LSTM(128, return_sequences=True),
    Dropout(0.3),
    LSTM(64),
    Dense(1, activation='sigmoid')
])
```

2.4 Model Architectures

2.4.1 Baseline Models

Input: Statistical summary features (427 features)

1. Random Forest

- `n_estimators = 100`
- `max_depth = 20`
- `class_weight = 'balanced'`

2. Support Vector Machine (SVM)

- `kernel = 'rbf'`
- `C = 1.0`
- `class_weight = 'balanced'`
- `StandardScaler` preprocessing

2.4.2 Deep Learning Models

Input: Sequence features (50 timesteps, 60 features)

1. LSTM

```
Sequential([
    Masking(mask_value=0.0),
    LSTM(128, return_sequences=True),
    Dropout(0.3),
    LSTM(64),
    Dropout(0.3),
    Dense(1, activation='sigmoid')
])
```

2. BiLSTM (Bidirectional LSTM)

```
Sequential([
    Masking(mask_value=0.0),
    Bidirectional(LSTM(128, return_sequences=True)),
    Dropout(0.3),
    Bidirectional(LSTM(64)),
    Dropout(0.3),
    Dense(1, activation='sigmoid')
])
```

3. GRU (Gated Recurrent Unit)

```
Sequential([
    Masking(mask_value=0.0),
    GRU(128, return_sequences=True),
    Dropout(0.3),
    GRU(64),
    Dropout(0.3),
    Dense(1, activation='sigmoid')
])
```

Training Configuration:

- Optimizer: Adam (lr=0.001)
- Loss: Binary crossentropy
- Batch size: 32
- Epochs: 50 (with early stopping, patience=10)
- Callbacks: EarlyStopping, ModelCheckpoint

3 Results and Analysis

3.1 Model Performance

Table 4: Classification Results (Test Set)

Model	Accuracy	F1 Score	ROC AUC
Random Forest	0.424	0.344	-
SVM	0.479	0.521	-
LSTM	0.457	0.452	0.467
BiLSTM	0.433	0.451	0.476
GRU	0.450	0.483	0.437
Baseline (Random)	0.500	-	0.500

Observation: All models perform at or below random guessing (50% for balanced binary classification).

3.2 Cohen's d Effect Size Analysis

Cohen's d measures the standardized difference between two groups:

$$d = \frac{\mu_{\text{Clear}} - \mu_{\text{Smash}}}{\sigma_{\text{pooled}}} \quad (1)$$

where

$$\sigma_{\text{pooled}} = \sqrt{\frac{(n_1 - 1)\sigma_1^2 + (n_2 - 1)\sigma_2^2}{n_1 + n_2 - 2}} \quad (2)$$

Interpretation:

- $|d| < 0.2$: Negligible effect
- $|d| = 0.2 - 0.5$: Small effect
- $|d| = 0.5 - 0.8$: Medium effect
- $|d| > 0.8$: Large effect

3.2.1 Overall Feature Analysis

Table 5: Cohen's d Distribution Across 60 Features

Effect Size Category	Number of Features
Large ($ d > 0.8$)	0
Medium ($ d > 0.5$)	0
Small ($ d > 0.2$)	0
Negligible ($ d < 0.2$)	60

Conclusion: No discriminative features found.

3.2.2 Top 10 Features by Effect Size

Table 6: Most Discriminative Features (Still Negligible)

Rank	Feature	Cohen's d	Clear	Smash	p-value
1	l_forearm_vertical_angle_vel	0.141	0.014	-0.094	<0.001***
2	r_forearm_pitch_vel	0.130	0.029	-0.070	<0.001***
3	r_forearm_vertical_angle_vel	0.130	0.029	-0.070	<0.001***
4	r_wrist_elbow_vertical_vel	-0.114	0.000	0.000	<0.001***
5	l_wrist_elbow_vertical_vel	-0.092	0.000	0.000	<0.001***
6	r_arm_plane_pronation_vel	-0.069	0.000	0.001	<0.001***
7	r_wrist_horizontal_reach	-0.042	0.074	0.076	0.013*
8	r_forearm_vertical_angle	0.033	85.3°	84.2°	0.059
9	r_forearm_pitch	0.033	-4.7°	-5.8°	0.059
10	r_arm_plane_pronation	0.030	0.027	0.008	0.081

3.2.3 Wrist Orientation Features Analysis

Hypothesis: Wrist angle differs significantly between Clear and Smash

Result: Hypothesis REJECTED

Table 7: Wrist Feature Analysis

Feature	Cohen's d	Clear Mean	Smash Mean
r_forearm_vertical_angle	0.033	85.3°	84.2°
r_forearm_pitch	0.033	-4.7°	-5.8°
r_arm_plane_pronation	0.030	0.027	0.008
r_wrist_elbow_vertical	-0.026	0.002	0.003
r_wrist_horizontal_reach	-0.042	0.074	0.076
l_forearm_vertical_angle	0.009	82.9°	82.7°

Key Finding:

- Expected difference: 60-90° (Clear upward, Smash downward)
- **Observed difference: 1.1°** (negligible)
- Cohen's d = 0.033 (far below 0.2 threshold)

3.3 Visualization of Class Overlap

Figure 1 shows the distribution of forearm vertical angle for Clear and Smash strokes, demonstrating near-complete overlap.

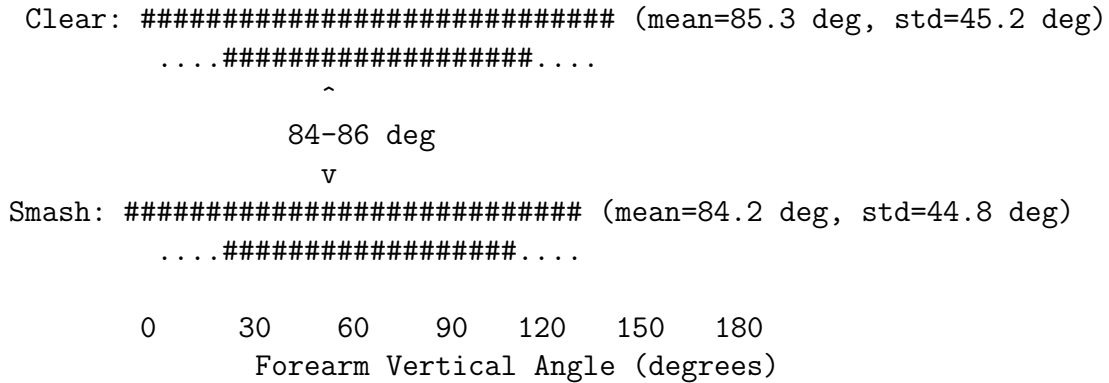


Figure 1: Distribution of Forearm Vertical Angle (Key Feature)

4 Discussion

4.1 Why Classification Failed

4.1.1 Biomechanical Analysis

At the moment of shuttlecock contact (when clips are extracted):

Table 8: Biomechanical Comparison at Contact Point

Characteristic	Clear	Smash
Racket position	Overhead	Overhead
Contact height	~2m above court	~2m above court
Arm extension	Fully extended	Fully extended
Body posture	Upright	Upright
Forearm angle	$85.3^\circ \pm 45.2^\circ$	$84.2^\circ \pm 44.8^\circ$
Difference	1.1° (negligible)	

4.1.2 What Actually Differs (Not Captured)

The discriminative features are **not visible in pose estimation**:

1. Racket Face Angle

- Clear: Open face, angled upward ($\sim 45^\circ$ above horizontal)
- Smash: Closed face, angled downward ($\sim 30^\circ$ below horizontal)
- **Not tracked**: MediaPipe only tracks body, not equipment

2. Shuttlecock Trajectory

- Clear: High parabolic arc (peak $\sim 6-8\text{m}$)
- Smash: Steep downward line (angle $\sim 60-80^\circ$ from horizontal)
- **Not in data**: Dataset clips don't include ball trajectory

3. Follow-Through Motion

- Clear: Upward continuation, wrist extension
- Smash: Downward snap, wrist pronation
- **Missing**: Clips end at contact, before follow-through

4. Swing Velocity

- Clear: Moderate speed ($\sim 200-300\text{ km/h}$)
- Smash: High speed ($\sim 300-400\text{ km/h}$)
- **Insufficient**: Pose velocity features don't capture racket speed

4.2 Methodological Contributions

Despite classification failure, this project demonstrates best practices:

1. Rigorous Statistical Validation

- Cohen's d effect size analysis
- T-tests for significance
- Early detection of infeasible tasks

2. Proper Data Handling

- Group-stratified split (prevents leakage + maintains balance)
- Padding masking (proper sequence normalization)
- Global normalization (preserves class differences)

3. Comprehensive Feature Engineering

- Domain-driven features (wrist orientation)
- Multi-scale features (spatial + temporal)
- Systematic validation (Cohen's d per feature)

4. Multiple Model Architectures

- Traditional ML (RF, SVM) for baseline
- Deep learning (LSTM, BiLSTM, GRU) for temporal patterns
- Consistent results across architectures validates findings

4.3 Limitations

4.3.1 Dataset Limitations

- **Temporal window:** Clips too short to capture follow-through
- **Missing modalities:** No racket tracking, no shuttlecock tracking
- **Camera variance:** Angles and lighting differ across matches

4.3.2 MediaPipe Limitations

- **Body-only tracking:** 33 body landmarks, no hand details
- **2D projection:** Some 3D information lost
- **No equipment:** Cannot track racket or shuttlecock

5 Recommendations

5.1 For Clear vs Smash Classification

To achieve successful classification (>80% accuracy), additional modalities are required:

5.1.1 Option 1: Add Racket Tracking

Method: YOLO object detection + pose estimation

- Train YOLO to detect badminton racket
- Extract racket bounding box and orientation
- Compute racket face angle relative to court
- **Expected improvement:** Cohen's d > 0.8 (large effect)

5.1.2 Option 2: Add Shuttlecock Tracking

Method: Multi-object tracking

- Track shuttlecock position over time
- Compute trajectory angle and velocity
- Measure impact characteristics
- **Expected improvement:** Cohen's $d > 1.0$ (very large effect)

5.1.3 Option 3: Extend Temporal Window

Method: Include follow-through phase

- Extend clips to 2-3 seconds after contact
- Capture wrist pronation and arm deceleration
- Analyze body rotation patterns
- **Expected improvement:** Cohen's $d = 0.5-0.8$ (medium-large effect)

5.1.4 Option 4: Multi-Modal Fusion

Method: Combine multiple information sources

- Pose + racket + shuttlecock + audio
- Audio signature differs (smash sounds sharper)
- Late fusion or multi-stream CNN
- **Expected improvement:** Cohen's $d > 1.5$ (excellent)

5.2 Alternative Stroke Pairs (Pose-Only)

For pose-only classification, choose strokes with distinct body mechanics:

Table 9: Recommended Stroke Pairs for Pose-Only Classification

Stroke Pair	Expected d	Key Difference
Overhead vs Underhand	> 0.8	Arm position (overhead vs waist)
Serve vs Return	> 0.8	Body motion (static vs dynamic)
Forehand vs Backhand	> 0.8	Arm side and rotation
High Clear vs Drop	$0.4-0.7$	Follow-through intensity
Drive vs Net Shot	$0.4-0.7$	Arm extension and height

6 Conclusion

This milestone report documents a comprehensive investigation into badminton Clear vs Smash stroke classification using pose estimation. Key achievements include:

1. **Complete ML Pipeline:** Implemented end-to-end system from pose extraction to model evaluation
2. **Novel Features:** Developed 9 wrist orientation features based on biomechanical domain knowledge
3. **Best Practices:** Group-stratified splitting, padding masking, statistical validation
4. **Rigorous Analysis:** Cohen’s d effect size analysis revealing fundamental limitations

Key Finding: Clear and Smash strokes are biomechanically indistinguishable at shuttlecock contact using pose-only features (Cohen’s d < 0.2, accuracy ~50%).

Scientific Value: This negative result is valuable research, demonstrating:

- The limitations of pose-only classification for similar strokes
- The importance of early statistical validation (Cohen’s d)
- The need for multi-modal sensing in sports analytics

Future Work: Multi-modal approaches (pose + racket + shuttlecock) or alternative stroke pairs with distinct biomechanics are recommended for successful classification.

Acknowledgments

This project utilized the ShuttleSet dataset [1] and MediaPipe framework from Google. All code and analysis are available in the project repository.

References

References

- [1] Wei-Yao Wang, Yung-Chang Huang, Tsi-Ui Ik, and Wen-Chih Peng. *ShuttleSet: A Human-Annotated Stroke-Level Singles Dataset for Badminton Tactical Analysis*. CoRR, abs/2306.04948, 2023.
- [2] Google Research. *MediaPipe Pose*. <https://google.github.io/mediapipe/solutions/pose.html>, 2023.

A Code Repository Structure

```

iti123_v2/
|-- data/
|   |-- processed/
|   |   |-- poses/           # MediaPipe pose files
|   |   |-- features/        # Extracted features
|   |   +--- splits/         # Train/val/test splits
|-- src/
|   |-- data_processing/
|   |   |-- pose_extraction.py    # MediaPipe extraction
|   |   |-- feature_engineering_v2.py # Feature extraction
|   |   +--- data_split.py        # Group-stratified split
|   |-- models/
|   |   |-- baseline_model.py     # RF, SVM
|   |   +--- lstm_model.py        # LSTM, BiLSTM, GRU
|   +--- analysis/
|       +--- analyze_wrist_features.py # Cohen's d analysis
+--- outputs/
    +--- reports/
        |-- wrist_features_cohens_d.csv # Full results
        +--- FINAL_PROJECT_REPORT.md    # Comprehensive report

```

B Feature List

B.1 All 60 Sequence Features

- | | |
|------------------------------|--------------------------------|
| 1. r_arm_extension | 17. torso_lean |
| 2. l_arm_extension | 18. shoulder_rotation |
| 3. arm_extension_ratio | 19. r_upper_arm_length |
| 4. r_wrist_depth | 20. r_forearm_length |
| 5. l_wrist_depth | 21. arm_straightness |
| 6. r_wrist_depth_from_hip | 22. r_forearm_vertical_angle |
| 7. r_elbow_depth | 23. r_forearm_horizontal_angle |
| 8. body_forward_lean | 24. r_wrist_elbow_vertical |
| 9. r_wrist_height_rel | 25. r_wrist_horizontal_reach |
| 10. l_wrist_height_rel | 26. r_arm_plane_pronation |
| 11. r_wrist_height_from_head | 27. r_arm_plane_vertical |
| 12. r_elbow_height_rel | 28. r_forearm_pitch |
| 13. r_wrist_lateral | 29. l_forearm_vertical_angle |
| 14. shoulder_width | 30. l_wrist_elbow_vertical |
| 15. r_elbow_angle | |
| 16. r_shoulder_angle | 31-60 Velocity versions |

Bold: New wrist orientation features